

SkinnedMeshRenderer 正確なインスタンス化ガイド

対象: VRM / FBX アバターのバイナリキャッシュシステム バージョン: 2026-01-26 プロジェクト: arCam / aiCam

1. 概要

SkinnedMeshRenderer (SMR) を正しくインスタンス化するには、以下の要素が**完全に整合**している必要がある：

要素	説明
bones[]	SMR に設定するボーン Transform 配列
bindposes[]	メッシュに保存されたバインドポーズ行列配列
boneWeights	各頂点のボーンインデックスとウェイト
meshRenderer.transform	SMR が付いている GameObject の Transform

これらのいずれか1つでも不整合があると、スキニングが崩壊する。

2. Bindpose の数学的定義

2.1 Unity の bindpose 定義

$$\text{bindpose}[i] = \text{bone}[i].\text{worldToLocalMatrix} \times \text{meshRenderer}.\text{localToWorldMatrix}$$

- **bone[i]** : i 番目のスキニングボーンの Transform
- **meshRenderer** : SkinnedMeshRenderer が付いている GameObject の Transform
- この式は**メッシュ作成時**（バインドタイム）の値で確定し、以後変化しない

2.2 bindpose の逆行列

$$\text{bindpose}[i]^{-1} = \text{meshRenderer}.\text{worldToLocalMatrix} \times \text{bone}[i].\text{localToWorldMatrix}$$

注意: $\text{bindpose}[i].\text{inverse}$ は $\text{bone}[i].\text{localToWorldMatrix}$ と等しくない。meshRenderer.worldToLocalMatrix が掛かっている。

2.3 正しいバインドタイム世界行列の算出

ボーン i のバインドタイム時の世界行列を求めるには：

$$\text{bone}[i].\text{localToWorldMatrix} \text{ (バインドタイム)} = \text{meshRenderer}.\text{localToWorldMatrix} \times \text{bindpose}[i]$$

2.4 よくある間違い

```
// [NG] 間違い: meshRenderer の Transform を無視  
bindTimeWorld = bindpose[i].inverse;
```

```
// [OK] 正解: meshRenderer の Transform を考慮  
bindTimeWorld = meshRenderer.localToWorldMatrix * bindpose[i].inverse;
```

meshRenderer が identity (位置 (0,0,0)、回転なし、スケール (1,1,1)) の場合のみ、`bindpose[i].inverse = bone[i].localToWorldMatrix` が成り立つ。VRM/Blender モデルでは meshRenderer に -90° X 回転が入ることが多い。

3. ランタイムスキニング計算

3.1 頂点変換の公式

```
vertex_world = SUM_i (weight_i * bone[i].localToWorldMatrix * bindpose[i] * vertex_meshLocal)
```

各頂点に対して、最大 4 つのボーンの影響を加重平均する。

3.2 バインドタイムでの検証

バインドタイム時 (ボーンが初期位置のとき) :

```
bone[i].localToWorldMatrix * bindpose[i]  
= bone[i].localToWorldMatrix * bone[i].worldToLocalMatrix * meshRenderer.localToWorldMatrix  
= meshRenderer.localToWorldMatrix
```

つまりバインドタイムでは、全頂点が `meshRenderer.localToWorldMatrix` だけで変換される。メッシュは meshRenderer の位置・回転で正しく表示される。

3.3 ポーズ変更時

アニメーションやリターゲットングでボーンが動くと :

`bone[i].localToWorldMatrix` $\neq$ バインドタイムの値

差分が頂点に反映され、メッシュが変形する。

4. データ構造と順序の厳密性

4.1 インデックスの一致が必須

```
mesh.bindposes[0] ↔ smr.bones[0] ↔ boneWeights.boneIndex0 = 0  
mesh.bindposes[1] ↔ smr.bones[1] ↔ boneWeights.boneIndex0 = 1  
...  
mesh.bindposes[N] ↔ smr.bones[N]
```

`bindpose[i]` は必ず `bones[i]` に対応する。この順序がずれるとスキニングが完全に崩壊する。

4.2 BoneWeight の構造

```
struct BoneWeight {  
    int boneIndex0, boneIndex1, boneIndex2, boneIndex3; // bones[] のインデックス  
    float weight0, weight1, weight2, weight3; // 各ボーンの影響度 (合計 = 1.0)  
}
```

boneIndex は `smr.bones[]` 配列のインデックスを参照する。bindposes と bones の順序が一致していないと、間違ったボーンの影響を受ける。

4.3 保存時のデータ

ファイル	内容	順序
<code>bones.json</code>	ボーン階層 (name, path, parentIndex, localTransform)	depth-first (親 → 子)
<code>meshes.bin</code>	メッシュデータ (頂点, bindposes, boneWeights)	bindpose 順序を保持
<code>smr.json</code>	SMR メタデータ (gameObjectPath, bonePaths[], materialNames)	bonePaths は bindpose 順序
<code>humanoid.json</code>	Humanoid マッピング (humanBoneName → bonePath)	HumanBodyBones 列挙順

5. VRM/FBX モデルの座標系

5.1 座標系の違い

ソフト	上方向	右手/左手
Unity	Y-up	左手系
Blender	Z-up	右手系
VRM (glTF)	Y-up	右手系

5.2 VRM/Blender モデルの典型的な構造

Blender でエクスポートされた VRM モデルでは、メッシュの頂点データが Z-up 座標系で作成されていることがある。この場合、meshRenderer に **-90° X 回転** が適用される：

```
VRM Root (identity)  
├─ Armature (identity)  
│   └─ Hips (Y=0.7, 立位の腰の高さ)  
│       └─ Spine  
│           └─ Chest → Neck → Head
```

```

├── Upper Leg.L → Lower Leg.L → Foot.L
├── Upper Leg.R → Lower Leg.R → Foot.R
├── Body (rotation: -90° X) ← meshRenderer
│   └── SkinnedMeshRenderer (Body メッシュ)
├── Face (rotation: -90° X) ← meshRenderer
│   └── SkinnedMeshRenderer (Face メッシュ)
└── Anchor (Y=0.638)

```

5.3 -90° X 回転の影響

```
meshRenderer.localToWorldMatrix = Rot_X(-90°)
```

このとき：

- $\text{bindpose}[i]^{(-1)} = \text{Rot_X}(+90^\circ) \times \text{bone}[i].\text{localToWorldMatrix}$
- $\text{bone}[i].\text{localToWorldMatrix} = \text{Rot_X}(-90^\circ) \times \text{bindpose}[i]^{(-1)}$

meshRenderer の -90° X 回転を無視すると：-Y 軸と Z 軸が入れ替わる - Hips が Y=0.7 ではなく Z=0.7 に配置される - アバターが下向きにインスタンスされる - BuildHumanAvatar が不正な骨格を受け取り、指の向きがおかしくなる

6. キャッシュ作成パイプライン

6.1 全体フロー

```
CreateCacheAsync(vrmPath, avatar)
```

- (1) 全ボーンの Transform をバックアップ


```

savedPositions[i] = allTransforms[i].localPosition
savedRotations[i] = allTransforms[i].localRotation
savedScales[i]    = allTransforms[i].localScale

```
- (2) Animator を無効化


```

animator.enabled = false

```

 (アニメーションによるボーン回転を防止)
- (3) ルートを原点に正規化


```

avatar.transform.localPosition = Vector3.zero
avatar.transform.localRotation = Quaternion.identity
avatar.transform.localScale    = Vector3.one

```
- (4) ボーンをバインドタイムにリセット ★重要


```

ResetBonesToBindTime(avatar)
├── 各 SMR から meshWorldMatrix = smr.transform.localToWorldMatrix を取得
├── bindTimeWorld = meshWorldMatrix × bindpose[i]^(-1)
├── localMatrix = parent.localToWorldMatrix^(-1) × bindTimeWorld
├── bone.localPosition = localMatrix の平行移動成分
└── bone.localRotation = localMatrix の回転成分

```

- (5) データを保存
 - └─ bones.json : ボーン階層 (バインドタイム状態の localTransform)
 - └─ humanoid.json : Humanoid マッピング
 - └─ meshes.bin : メッシュ (bindposes, boneWeights 含む)
 - └─ smr.json : SMR メタデータ (bonePaths[])
 - └─ materials.json: マテリアル
 - └─ textures/ : テクスチャ画像
- (6) 整合性検証
 - VerifyBindposeBoneConsistency(smrs)
 - └─ 各ボーンの実際の位置と bindpose から算出した位置を比較
- (7) 全ボーンの Transform を復元 (元のゲームに影響しない)
 - allTransforms[i].localPosition = savedPositions[i]
 - allTransforms[i].localRotation = savedRotations[i]
 - allTransforms[i].localScale = savedScales[i]
- (8) Animator を復元
 - animator.enabled = animatorWasEnabled

6.2 ResetBonesToBindTime の処理順序

親 → 子の順序 (depth-first) で処理する理由:

bindTimeWorld[Hips] が確定

- Spine の localMatrix = Hips.localToWorldMatrix⁽⁻¹⁾ × bindTimeWorld[Spine]
- Chest の localMatrix = Spine.localToWorldMatrix⁽⁻¹⁾ × bindTimeWorld[Chest]
- ...

親の世界行列が先に確定していないと、子の localMatrix を正しく計算できない。GetComponentsInChildren<Transform>() は depth-first 順序を返すため、自然に親 → 子の処理順序が保証される。

7. キャッシュロードパイプライン

7.1 全体フロー

LoadFromCacheAsync(cacheId)

STEP 1: ボーン階層を復元

- BoneHierarchyCacheSerializer.Reconstruct(boneCache)
 - └─ 全 BoneInfo の GameObject を作成
 - └─ 保存時の localPosition/Rotation/Scale をそのまま適用
 - └─ SetParent(parent, worldPositionStays: false) で階層構築

STEP 2: Humanoid データを読み込み (まだ適用しない)

humanoidCache = HumanoidCacheSerializer.DeserializeFromJson(json)

STEP 3: メッシュを復元

```
meshes = MeshCacheSerializer.DeserializeFromBinary(meshesPath)
└─ bindposes, boneWeights を元の順序で復元
```

STEP 4: BlendShape を適用

```
BlendShapeCacheSerializer.DeserializeAndApply(blendShapesPath, meshes)
```

STEP 5: テクスチャをロード

```
textures = TextureCacheManager.LoadTextureAsync(textureId)
```

STEP 6: マテリアルを復元

```
materials = MaterialCacheSerializer.Reconstruct(materialCache, textures)
```

STEP 6.5: SMR メタデータをロード

```
smrCache = SkinnedMeshRendererCacheSerializer.DeserializeFromJson(json)
```

STEP 7: ルートを原点にリセット ★重要

```
avatar.transform.localPosition = Vector3.zero
avatar.transform.localRotation = Quaternion.identity
avatar.transform.localScale    = Vector3.one
(bindposes はアバターが原点にある状態で計算されているため)
```

STEP 8: SMR をセットアップ ★Animator より先に実行

```
SetupSkinnedMeshRenderers(avatar, meshes, materials, smrCache)
└─ 各 SMR 情報に対して:
    └─ gameObjectPath で GameObject を検索 (なければ作成)
    └─ SMR コンポーネントを取得または追加
    └─ smr.sharedMesh = mesh
    └─ smr.bones = BuildBoneArray(avatar, bonePaths)
    └─ 各 bonePath に対して avatar.transform.Find(path) で Transform を取得
        └─ 順序は bindpose 順序と一致 ★
    └─ smr.rootBone = FindTransformByPath(avatar, rootBonePath)
    └─ smr.sharedMaterials = マテリアル配列
└─ この時点でスキニングが正しく動作する状態
```

STEP 9: Humanoid Avatar を作成 ★SMR の後に実行

```
HumanoidCacheSerializer.CreateAvatar(humanoidCache, avatar)
└─ 全 Transform → SkeletonBone[] に変換
└─ Humanoid マッピング → HumanBone[] に変換
└─ HumanDescription を構築 (twist, stretch, limits)
└─ AvatarBuilder.BuildHumanAvatar(root, humanDescription)
    └─ ボーン Transform を変更する可能性あり (リターゲットイング)
avatar.animator = humanAvatar
```

Return: avatar (完全に再構築されたアバター)

7.2 Step 8 → Step 9 の順序が重要な理由

SMR セットアップ前に `BuildHumanAvatar` を呼ぶと：

`BuildHumanAvatar` がボーン `Transform` を変更（リターゲットイング）

→ ボーンの位置が `bindpose` 作成時の位置からずれる

→ `SMR.bones` の位置 $\neq$ `bindpose` が想定する位置

→ スキニングが崩壊

正しい順序：

1. SMR セットアップ（`bindpose` と `bones` が一致する状態で完了）
2. `BuildHumanAvatar`（ボーンを変更してもスキニングは既にセットアップ済み）
3. ランタイムスキニングが差分を正しく計算

8. Humanoid Avatar の構築

8.1 HumanDescription の構成

```
HumanDescription {
    SkeletonBone[] skeleton; // 全ボーンの名前と localTransform
    HumanBone[] human;      // HumanBodyBones → ボーン名のマッピング

    // 補間パラメータ
    float upperArmTwist;     // 上腕のツイスト補間 (0-1)
    float lowerArmTwist;    // 下腕のツイスト補間 (0-1)
    float upperLegTwist;    // 上脚のツイスト補間 (0-1)
    float lowerLegTwist;   // 下脚のツイスト補間 (0-1)
    float armStretch;       // 腕のストレッチ (0-1)
    float legStretch;       // 脚のストレッチ (0-1)
    float feetSpacing;      // 足の間隔
    bool hasTranslationDoF; // 平行移動の自由度
}
```

8.2 SkeletonBone

```
SkeletonBone {
    string name;           // ボーン名 (Transform.name)
    Vector3 position;      // localPosition
    Quaternion rotation;   // localRotation
    Vector3 scale;         // localScale
}
```

全ての `Transform`（ボーン以外の `meshRenderer` なども含む）が `skeleton` に含まれる。`BuildHumanAvatar` は `HumanBone` に記載されたボーンのみ処理する。

8.3 HumanBone

```
HumanBone {
    string humanName;      // Unity の Humanoid ボーン名 (例: "LeftUpperArm")
```

```

    string boneName;           // skeleton 内のボーン名 (例: "Upper Arm.L")
    HumanLimit limit;          // 関節の可動範囲
}

```

8.4 ボーン名の変換

HumanBodyBones 列挙型の名前と HumanTrait の名前は一部異なる：

HumanBodyBones	HumanTrait 名
LeftThumbProximal	"Left Thumb Proximal"
LeftIndexProximal	"Left Index Proximal"
RightLittleDistal	"Right Little Distal"
LeftUpperArm	"LeftUpperArm" (同じ)

指ボーンのみスペース区切りに変換が必要。

8.5 BuildHumanAvatar の副作用

AvatarBuilder.BuildHumanAvatar は以下の副作用を持つ：

1. **ボーン Transform の変更**: リターゲットングにより、一部のボーンの localRotation が変更される
2. **T-ポーズへの正規化**: ボーンの向きを Unity の Humanoid テンプレートに合わせる
3. **肩・腕の回転**: 特に肩と上腕のボーンが回転されることが多い

これらの変更は、キャッシュ保存時に **ResetBonesToBindTime** で元に戻され、キャッシュロード時に **BuildHumanAvatar** で再適用される。

9. FBX 固有の注意点

9.1 FBX と VRM の違い

項目	VRM	FBX
座標系	Y-up (glTF 規格)	ファイル内で指定可能 (多くは Z-up)
meshRenderer 回転	-90° X が一般的	インポーターに依存
Humanoid 設定	VRM メタデータに含む	FBX ファイルには含まない
ボーン命名	VRM 規格に準拠	モデラーに依存
スケール	メートル単位	cm/m が混在

9.2 FBX インポート時の確認事項

1. **meshRenderer の Transform を確認**
 - **smr.transform.localToWorldMatrix** が identity でない場合、bindpose 計算に影響
 - Blender エクスポートの FBX では -90° X 回転が一般的

2. スケールファクター

- FBX のスケールファクター (0.01 = cm → m 変換) が適用されている場合、全ボーンの localScale が (0.01, 0.01, 0.01) になる可能性
- bindpose にもスケールが反映されるため、整合性は保たれる

3. ボーン命名規則

- Humanoid マッピングはボーン名で行う
- FBX のボーン名は Blender/Maya/3ds Max の命名に依存
- 自動マッピングが必要な場合がある

9.3 FBX 対応時の実装チェックリスト

- ☐ meshRenderer の Transform を **ResetBonesToBindTime** で考慮している
- ☐ bindpose のインデックス順序が bones[] と一致している
- ☐ スケールファクターが正しく処理されている
- ☐ Humanoid ボーン名のマッピングが FBX の命名規則に対応している
- ☐ **VerifyBindposeBoneConsistency** で整合性検証が通る
- ☐ BuildHumanAvatar の前に SMR セットアップが完了している

10. トラブルシューティング

10.1 スキニングが崩壊する場合

症状	原因	解決策
メッシュが大きく歪む	bones[] と bindposes[] の順序不一致	bonePaths の順序を bindpose 順序に合わせる
アバターが下向き	meshRenderer の回転を無視	meshWorldMatrix × bindpose[i]^(-1) を使用
肩が内側に窄む	BuildHumanAvatar の二重適用	ResetBonesToBindTime でバインドタイムに戻してから保存
指が振れる	不正な骨格で BuildHumanAvatar	meshRenderer Transform を考慮して正しいバインドタイム状態を保存
メッシュが原点に集まる	ルートが原点にない	SMR セットアップ前にルートを原点にリセット
メッシュが表示されない	bones[] に null が含まれる	BuildBoneArray のログで missing bones を確認

10.2 デバッグ方法

1. bindpose-bone 整合性検証:

```
var meshWorldMatrix = smr.transform.localToWorldMatrix;
var bindTimeWorld = meshWorldMatrix * bindposes[i].inverse;
var expectedPos = new Vector3(bindTimeWorld.m03, bindTimeWorld.m13, bindTimeWorld.m23);
var actualPos = bones[i].position;
float error = Vector3.Distance(expectedPos, actualPos);
// error > 0.01 なら不整合
```

2. ボーン配列の検証:

```
Debug.Log($"bindposes: {mesh.bindposes.Length}, bones: {smr.bones.Length}");
// 数が一致しない場合は致命的
for (int i = 0; i < smr.bones.Length; i++) {
    if (smr.bones[i] == null)
        Debug.LogError($"Bone [{i}] is null!");
}
```

3. meshRenderer Transform の確認:

```
var smr = avatar.GetComponentInChildren<SkinnedMeshRenderer>();
Debug.Log($"SMR transform: pos={smr.transform.localPosition}, " +
    $"rot={smr.transform.localRotation.eulerAngles}, " +
    $"scale={smr.transform.localScale}");
// identity でない場合は bindpose 計算に影響
```

11. データフロー図

11.1 キャッシュ作成フロー

VRM/FBX ファイル

↓ (UniVRM / Assimp でロード)

GameObject (ボーン階層 + SMR + Animator)

↓

↓ (1) Animator 無効化

↓ (2) ルートを原点に正規化

↓ (3) ResetBonesToBindTime

↓ └─ meshWorldMatrix × bindpose[i]^(-1) → バインドタイム世界行列

↓ └─ parent^(-1) × バインドタイム世界行列 → localTransform

↓

ボーンがバインドタイム状態

↓

└─ bones.json (localPosition, localRotation, localScale)

└─ meshes.bin (bindposes, boneWeights, 頂点データ)

└─ smr.json (bonePaths[], gameObjectPath, rootBonePath)

└─ humanoid.json (humanBoneName → bonePath)

└─ materials.json

└─ textures/

└─ manifest.json

11.2 キャッシュロードフロー

キャッシュファイル群

↓

└─ bones.json → BoneHierarchyCacheSerializer.Reconstruct

└─ └─ GameObject 階層作成 (バインドタイム localTransform)

```

├── meshes.bin → MeshCacheSerializer.Deserialize
│   └── Mesh 復元 (bindposes + boneWeights 順序保持)
├── smr.json → SkinnedMeshRendererCacheSerializer
│   └── bonePaths[] で bones[] 配列を構築
├── materials.json + textures/ → Material 復元
├── humanoid.json → HumanoidCacheSerializer
↓
↓ STEP 7: ルートを原点にリセット
↓ STEP 8: SMR セットアップ (bones = BuildBoneArray(bonePaths))
↓ STEP 9: BuildHumanAvatar (Humanoid リターゲットイング)
↓
完成したアバター GameObject
↓
↓ 位置・回転を復元 (ApplyTransform)
↓ メモリキャッシュに登録
↓
シーンに配置

```

12. 参照ファイル一覧

ファイル	役割	重要な行
AvatarCacheManager.cs	キャッシュ作成・ロードの中核	ResetBonesToBindTime (646-708), LoadFromCacheAsync (282-414) Reconstruct (88-127)
BoneHierarchyCacheSerializer.cs	ボーン階層の保存・復元	
HumanoidCacheSerializer.cs	Humanoid マッピング保存・Avatar 構築	CreateAvatar (81-162)
MeshCacheSerializer.cs	メッシュの保存・復元 (bindposes 含む)	WriteMesh (109-211), ReadMesh (213-344)
SkinnedMeshRendererCacheSerializer.cs	SMR メタデータ保存・ボーン配列構築	BuildBoneArray (169-197)
BlendShapeCacheSerializer.cs	BlendShape の保存・復元	-
MaterialCacheSerializer.cs	マテリアルの保存・復元	-
AvatarCacheIntegrator.cs	キャッシュシステムとスロットシステムの統合	-
AvatarMemoryCache.cs	メモリキャッシュ (LRU) 管理	SwitchToSlotAsync (403-619)

13. 過去のバグと修正履歴

このプロジェクトで発生した SMR/スキニング関連のバグを時系列で記録する。将来 FBX 対応やキャッシュシステムの改修を行う際の参考にすること。

13.1 肩から先の破綻 (BuildHumanAvatar の二重適用)

報告時期: 2026-01 (Issue #416) **症状:** キャッシュから復元したアバターの肩が内側に窄み、上腕が不自然に回転する

原因 VRM を通常ロードする際のパイプラインは以下の通り：

【通常ロード】

VRM ファイル → UniVRM パース → バインドタイム状態のボーン
→ BuildHumanAvatar (1回目) → リターゲットイング適用 → 正しいアバター

旧キャッシュ実装では、BuildHumanAvatar が適用された後のボーン状態をそのまま保存していた：

【旧キャッシュ作成】

通常ロード済みアバター (BuildHumanAvatar 適用済み)
→ ボーンの localTransform をそのまま保存
(肩・上腕が既にリターゲットイングで回転された状態)

【旧キャッシュロード】

保存データ → ボーン階層復元 (リターゲットイング済みの値)
→ BuildHumanAvatar (2回目) → 既にリターゲットイング済みの値に対して再度リターゲットイング
→ 肩・上腕が二重に回転 → 破綻

BuildHumanAvatar が行うこと AvatarBuilder.BuildHumanAvatar は、入力されたボーンの向きを Unity の Humanoid テンプレート (T-ポーズ) に正規化するため、一部のボーンの localRotation を変更する。特に肩と上腕のボーンは、元のモデルのポーズと Unity テンプレートの差が大きいため、回転の変更量が大きい。

この「変更」を 2 回適用すると、差分が 2 倍になり、肩が内側に窄んだような見た目になる。

修正内容 キャッシュ作成時に ResetBonesToBindTime を呼び、ボーンをバインドタイム状態に戻してから保存するように変更した。これにより、通常ロードと同じパイプラインが再現される：

【修正後キャッシュ作成】

通常ロード済みアバター (BuildHumanAvatar 適用済み)
→ ResetBonesToBindTime (bindpose からバインドタイム状態を再計算)
→ ボーンの localTransform を保存 (バインドタイム状態)

【修正後キャッシュロード】

保存データ → ボーン階層復元 (バインドタイム状態)

→ BuildHumanAvatar (1回目) → リターゲティング適用 → 正しいアバター

教訓

- キャッシュに保存するボーン状態は、必ず**バインドタイム** (bindpose が作成された時点) でなければならない
- BuildHumanAvatar はべき等ではない (2回適用すると結果が変わる)
- 「保存 → 復元」のサイクルでは、元のロードパイプラインと同じ入力を再現することが重要

13.2 アバターが下向き + 指の振れ (meshRenderer Transform の無視)

報告時期: 2026-01 (Issue #416 追加報告) **症状:** (1) アバターが顔面を下に向けた状態でインスタンスされる (2) 親指が内側に振れる

原因 13.1 の修正で **ResetBonesToBindTime** を追加したが、bindpose の逆行列をそのままボーンのワールド行列として使用していた。これは meshRenderer の Transform が identity の場合にのみ正しい。

// [NG] 旧実装

```
bindTimeWorldMatrices[bones[i]] = bindposes[i].inverse;
```

VRM モデルの meshRenderer (Body, Face 等) には **-90° X 回転** が適用されており、bindpose は以下の式で定義される:

```
bindpose[i] = bone[i].worldToLocalMatrix × meshRenderer.localToWorldMatrix
```

したがって:

```
bindpose[i]^(-1) = meshRenderer.worldToLocalMatrix × bone[i].localToWorldMatrix
```

bindpose[i].inverse を直接ワールド行列として使うと、meshRenderer の -90° X 回転の逆変換 (+90° X 回転) が余分に含まれる。結果:

項目	期待値	実際の値 (バグ時)
Hips Y 座標	0.700 (腰の高さ)	0.000
Hips Z 座標	0.000	0.700 (前方にずれる)
Hips X 回転	0°	90° (前傾)
全体の見た目	直立	顔面を下に向けた状態

この不正な骨格を BuildHumanAvatar に渡すと、指のリターゲティングも不正になり、親指が内側に振れる。

修正内容 **meshRenderer.localToWorldMatrix** を掛けて meshRenderer の寄与を打ち消す:

// [OK] 修正後

```
var meshWorldMatrix = smr.transform.localToWorldMatrix;
```

```
bindTimeWorldMatrices[bones[i]] = meshWorldMatrix * bindposes[i].inverse;
```

数学的検証:

```

meshWorldMatrix × bindpose[i]^(-1)
= meshRenderer.localToWorldMatrix × meshRenderer.worldToLocalMatrix × bone[i].localToWorldMatrix
= I × bone[i].localToWorldMatrix
= bone[i].localToWorldMatrix (バインドタイム)

```

meshRenderer の Transform が完全に打ち消され、純粋なボーンのワールド行列が得られる。

meshRenderer に回転がある理由 VRM/Blender モデルでは、メッシュの頂点データが Z-up 座標系で作成されていることがある。Unity の Y-up 座標系に合わせるため、meshRenderer の GameObject に -90° X 回転を適用して座標変換を行う。ボーン階層自体は Y-up で構築されているため、回転は不要。

Armature (identity) — ボーンは Y-up

Body (rot: -90° X) — メッシュ頂点を Z-up → Y-up に変換

bindpose はこの -90° X 回転を内部に取り込んでいるため、bindpose の逆行列を使う際には必ず meshRenderer.localToWorldMatrix を掛けて打ち消す必要がある。

教訓

- `bindpose[i].inverse` はボーンのワールド行列と等しくない
- meshRenderer の Transform が identity であることを仮定してはならない
- 公式の定義 `bindpose[i] = bone[i].W2L × mesh.L2W` から導出すれば間違いない
- VRM/FBX とともに meshRenderer に回転が入る可能性があるため、常に考慮すること

13.3 修正の適用箇所まとめ

両方のバグ修正は AvatarCacheManager.cs の以下 2 メソッドに適用された：

メソッド	修正内容
ResetBonesToBindTime	<code>meshWorldMatrix × bindpose[i]^(-1)</code> でバインドタイム世界行列を算出
VerifyBindposeBoneConsistency	同上の式で整合性検証時の期待位置を算出

修正前後のコード差分：

```
// === ResetBonesToBindTime ===
```

```
// 修正前 (13.1 時点: ResetBonesToBindTime 追加、13.2 バグあり)
```

```
bindTimeWorldMatrices[bones[i]] = bindposes[i].inverse;
```

```
// 修正後 (13.2 修正適用)
```

```
var meshWorldMatrix = smr.transform.localToWorldMatrix;
```

```
bindTimeWorldMatrices[bones[i]] = meshWorldMatrix * bindposes[i].inverse;
```

```
// === VerifyBindposeBoneConsistency ===
```

```
// 修正前
var bindposeInv = bindposes[i].inverse;
var bpPos = new Vector3(bindposeInv.m03, bindposeInv.m13, bindposeInv.m23);

// 修正後
var meshWorldMatrix = smr.transform.localToWorldMatrix;
var bindTimeWorld = meshWorldMatrix * bindposes[i].inverse;
var bpPos = new Vector3(bindTimeWorld.m03, bindTimeWorld.m13, bindTimeWorld.m23);
```